

R-Control IP4 – UDP Legacy API

Was ist das?

Eine sehr einfache UDP-Schnittstelle: Sie schicken eine Textzeile, das Gerät antwortet mit einer Textzeile. Kein Verbindungsaufbau, kein Anmelden.

Diese Schnittstelle ist für bestehende Installationen gedacht. Für neue Projekte nehmen Sie besser die [HTTP/JSON-API](#) oder [Modbus TCP](#).

Wichtig: Keine Anmeldung, keine Verschlüsselung. Nur in einem abgeschotteten Netz einsetzen.

In 2 Minuten zum Ziel

Schritt 1: Einschalten

Weboberfläche → **Einstellungen** → **Schnittstellen** → **UDP Legacy API** → Haken setzen, Port prüfen (Standard **30303**), speichern.

Schritt 2: Die Befehle

Befehl	Wirkung	Antwort
OUT1 1	Ausgang 1 einschalten	OK
OUT1 0	Ausgang 1 ausschalten	OK
OUT1 2	Ausgang 1 umschalten	OK
OUT1 ?	Zustand abfragen	0 oder 1
T ?	Temperatur abfragen	z. B. 21.40
OUT1 IMP 00:00:03 1	Impuls EIN für 3 Sekunden	OK

Die Ausgänge sind hier **1-basiert**: OUT1 bis OUT4.

Schritt 3: Ausprobieren

Unter Linux/macOS mit **netcat**:

```
echo "OUT1 1" | nc -u -w1 192.168.0.3 30303
echo "T ?"    | nc -u -w1 192.168.0.3 30303
```

Unter Windows mit PowerShell:

```
$client = [System.Net.Sockets.UdpClient]::new()
$daten  = [Text.Encoding]::ASCII.GetBytes("OUT1 1")
```

```
$client.Send($daten, $daten.Length, "192.168.0.3", 30303)
```

Ab hier folgt der **vollständige Referenzteil** mit allen Details.

Diese Dokumentation beschreibt die **UDP Legacy API** des R-Control IP4 aus Integrationsicht. Sie richtet sich an Kunden/Partner, die die Schnittstelle in ein eigenes System (SPS/SCADA, Gebäudeautomation, Middleware, Testsysteme) einbinden möchten.

Stand: Implementierung in `components/udp_api_legacy` (Firmware-Repo). Die UDP Legacy API ist bewusst **minimalistisch** und **nicht authentifiziert**.

1. Überblick

Zweck

- Einfache UDP-basierte Steuerung/Abfrage von Relais-Ausgängen.
- Abfrage der letzten Temperaturmessung (NTC-Sensor), sofern verfügbar.

Eigenschaften

- Transport: UDP/IPv4
- Nutzdaten: ASCII-Text, zeilenbasiert
- Zustandslos (pro Datagramm), keine Sessions
- **Keine Authentifizierung / keine Verschlüsselung** (LAN/isoliertes Netz vorausgesetzt)

2. Aktivierung & Konfiguration

Die Schnittstelle wird über die Gerätekonfiguration aktiviert.

2.1 Konfigurationsschlüssel

- `config.interfaces.udp_api_legacy.enabled` (bool)
 - `true` = UDP-Schnittstelle aktiv
 - `false` = deaktiviert (Default im Auslieferungszustand typischerweise `false`)
- `config.interfaces.udp_api_legacy.udp_port` (int)
 - UDP-Port des Servers

2.2 Default-Port

- In der Auslieferungs-Konfigurationsdatei (Beispiel `spiffs/config.json`) ist oft `udp_port: 30303` hinterlegt.
- Falls der Schlüssel **nicht** gesetzt ist, nutzt die Firmware als Fallback **Port 12345**.

Empfehlung: Port **explizit** in der Konfiguration setzen.

2.3 mDNS/Service Discovery

Wenn mDNS aktiv ist, registriert das Gerät bei aktivierter UDP Legacy API den Dienst:

- Service: `_udp_api_legacy._udp`

- Port: entspricht `config.interfaces.udp_api_legacy.udp_port`

Zusätzlich kann der `_http._tcp`-Service TXT-Records enthalten (`udp_api_legacy=true/false`, `udp_port=<port>`), um Integratoren die Port-Ermittlung zu erleichtern.

3. Transport & Datenformat

3.1 UDP-Socket

- Protokoll: UDP (Datagramme)
- Bind: `0.0.0.0:<udp_port>` (INADDR_ANY)
- IPv4: ja
- IPv6: nein (der Socket wird als AF_INET erstellt)

3.2 Encoding / Zeilenverarbeitung

- Zeichensatz: ASCII (druckbare Zeichen), Groß-/Kleinschreibung teils tolerant (siehe Kommandos)
- Ein Datagramm kann **eine oder mehrere Befehlszeilen** enthalten.
 - Zeilentrenner: `\n`
 - Optionales `\r` am Zeilenende wird toleriert (CRLF möglich).
- Whitespace am Zeilenanfang/-ende wird entfernt.

3.3 Maximale Datagrammgröße

- Empfangspuffer: **192 Bytes Nutzdaten** pro UDP-Datagramm.
- Größere Datagramme werden abgeschnitten bzw. führen zu unvollständigen Zeilen → vermeiden.

3.4 Antwortverhalten

- Für **jede** gültige Befehlszeile sendet das Gerät **eine** Antwort (UDP-Datagramm) an die Quelladresse.
- Enthält ein Datagramm mehrere Zeilen, folgen entsprechend mehrere Antwortdatagramme.

4. Adressierung von Relais-Kanälen

Die Relais werden als Kanäle `OUT<n>` adressiert:

- `<n>` ist **1-basiert**: `OUT1`, `OUT2`, ...
- Gültiger Bereich: `1 .. N`
 - `N` entspricht der Anzahl `config.peripherals.switch_outputs[]`.
 - Fallback: Wenn die Konfiguration nicht lesbar ist, wird `N=4` angenommen.

Ungültige Kanäle werden mit `ERROR: Invalid channel` beantwortet.

5. Kommandos

Allgemeine Syntax

- Tokens werden durch ein einzelnes oder mehrere Leerzeichen getrennt.
- Unbekannte Kommandos werden abgewiesen.

5.1 Temperatur abfragen

Request

- `T ?`

Response

- Erfolgreich: `<tempC>\n` (Float mit 2 Nachkommastellen, z. B. `22.50\n`)

Fehlerfälle

- `ERROR: Sensor not ready\n` – noch keine gültige Messung verfügbar
- `ERROR: Sensor fault\n` – Sensorstatus fehlerhaft

5.2 Relaiszustand abfragen

Request

- `OUT<n> ?`

Response

- `0\n` – Relais aus
- `1\n` – Relais ein

5.3 Relais schalten (Set/Reset/Toggle)

Request

- `OUT<n> 1` – einschalten
- `OUT<n> 0` – ausschalten
- `OUT<n> 2` – toggeln

Response

- Erfolgreich: `OK\n`

5.4 Impuls auslösen

Startet einen zeitlich begrenzten Impuls am Ausgang.

Request

- `OUT<n> IMP HH:MM:SS 1` – Impuls EIN für Dauer
- `OUT<n> IMP HH:MM:SS 0` – Impuls AUS für Dauer

Parameter

- Dauerformat: `HH:MM:SS`
 - `MM` und `SS` müssen im Bereich `0..59` liegen
 - Gesamtzeit muss `> 0` sein

Response

- Erfolgreich: `OK\n`

Fehlerfälle

- `ERROR: Missing parameter\n` – z. B. zu wenige Tokens
- `ERROR: Invalid duration\n` – ungültiges Zeitformat oder `00:00:00`
- `ERROR: Invalid parameter\n` – letzter Parameter nicht `0/1`

6. Response-Codes & Fehlertexte

Die UDP Legacy API nutzt einfache Textantworten.

6.1 Erfolgsantwort

- `OK\n`

6.2 Datenantwort

- Temperatur: z. B. `22.50\n`
- Relaisstatus: `0\n` oder `1\n`

6.3 Fehlerantworten (aus Firmware)

Antwort	Typischer Auslöser
<code>ERROR: Unknown command\n</code>	Unbekannter Befehl, falsche Tokenanzahl, falsches Format
<code>ERROR: Invalid channel\n</code>	<code>OUT</code> fehlt, Kanal nicht numerisch, oder außerhalb <code>1..N</code>
<code>ERROR: Missing parameter\n</code>	Impuls-Kommando ohne alle Parameter
<code>ERROR: Invalid duration\n</code>	Dauer nicht <code>HH:MM:SS</code> , Minuten/Sekunden ≥ 60 , oder <code>0</code>
<code>ERROR: Invalid parameter\n</code>	Impuls-Level nicht <code>0</code> oder <code>1</code>
<code>ERROR: Internal error\n</code>	Unerwarteter Fehler beim Ausführen
<code>ERROR: Sensor not ready\n</code>	Temperaturabfrage bevor Messung verfügbar ist
<code>ERROR: Sensor fault\n</code>	Sensorstatus ungültig

Hinweis: Alle Antworten enden mit `\n`.

7. Integrationshinweise (Best Practices)

7.1 UDP-Charakteristik

UDP ist nicht verbindungsorientiert. Für eine robuste Integration:

- Mit **Timeouts** arbeiten (z. B. 200–1000 ms, abhängig vom Netzwerk)
- Bei fehlender Antwort **Retry** (z. B. 1–3 Versuche)
- Antwort eindeutig der Anfrage zuordnen (z. B. pro Anfrage genau ein Kommando senden)

7.2 Idempotenz

- Idempotent (gut für Retries):

- `OUT<n> 1`, `OUT<n> 0`, `OUT<n> ?, T ?`
- Nicht-idempotent (Retries können Nebenwirkungen erzeugen):
 - `OUT<n> 2` (Toggle)
 - `OUT<n> IMP ...` (kann mehrfach triggern)

Empfehlung: Bei kritischen Anwendungen Toggle vermeiden und stattdessen Zustand abfragen (`OUT<n> ?`) und danach gezielt `0/1` setzen.

7.3 Sequenzierung / Mehrzeilen-Datagramme

Mehrere Zeilen pro Datagramm sind möglich, jedoch werden Antworten als einzelne Datagramme gesendet. Empfehlung für Integratoren:

- Pro UDP-Datagramm **genau ein Kommando** senden (vereinfacht Matching/Retry-Logik).

8. Security-Hinweise

Diese Schnittstelle ist **nicht authentifiziert** und kann Relais schalten.

Empfohlene Schutzmaßnahmen:

- Nur in **vertrauenswürdigen Netzen** nutzen (OT-/LAN-Segment)
- Zugriff per **Firewall/VLAN** einschränken (nur definierte Quell-IP/Hosts)
- UDP Legacy API im Normalbetrieb deaktiviert lassen (`enabled=false`) und nur bei Bedarf aktivieren

9. Beispiele

9.1 Beispiel (Python, plattformneutral)

Sendet ein Kommando und wartet auf eine Antwort.

```
import socket

DEVICE_IP = "192.168.0.3"
PORT = 30303
TIMEOUT_S = 1.0

def udp_request(cmd: str) -> str:
    data = (cmd.strip() + "\n").encode("ascii")
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(TIMEOUT_S)
        s.sendto(data, (DEVICE_IP, PORT))
        resp, _ = s.recvfrom(512)
        return resp.decode("ascii", errors="replace")

print("OUT1 ? =>", udp_request("OUT1 ?"))
print("OUT1 1 =>", udp_request("OUT1 1"))
print("T ?   =>", udp_request("T ?"))
```

9.2 Beispiel (Relais gezielt setzen statt Toggle)

1. Zustand lesen: `OUT2 ?`
2. Wenn `0`, dann setzen: `OUT2 1`

9.3 Beispiel (Impuls 10 Sekunden EIN)

- `OUT3 IMP 00:00:10 1`

10. Kompatibilität & Änderungen

Die UDP Legacy API ist eine Bestands-/Legacy-Schnittstelle und kann in zukünftigen Firmware-Versionen erweitert oder angepasst werden.

Für Projekte mit langfristigem Wartungsbedarf wird empfohlen, alternativ die authentifizierte JSON-REST-API oder das WebSocket-Gateway zu verwenden (siehe [docs/API.md](#)).